

0.1 System Architecture Design and Integration Workflow

0.1.1 Overall System Architecture

The proposed system is designed based on an Agentic-SOAR architecture with a strong focus on automated self-healing and secure vulnerability remediation. Instead of limiting security operations to detection and alert generation, the system is designed to support the full lifecycle of incident handling, including vulnerability analysis, patch generation, validation, administrator approval, and secure deployment. The entire infrastructure is deployed on Docker to ensure portability, isolation, scalability, and simplified management across different operational environments.

The architecture is divided into three major logical zones: the Target Zone, the SOC/SOAR Control Plane, and External Services. Each zone performs a dedicated responsibility while maintaining tightly controlled communication channels to support both security and operational efficiency.

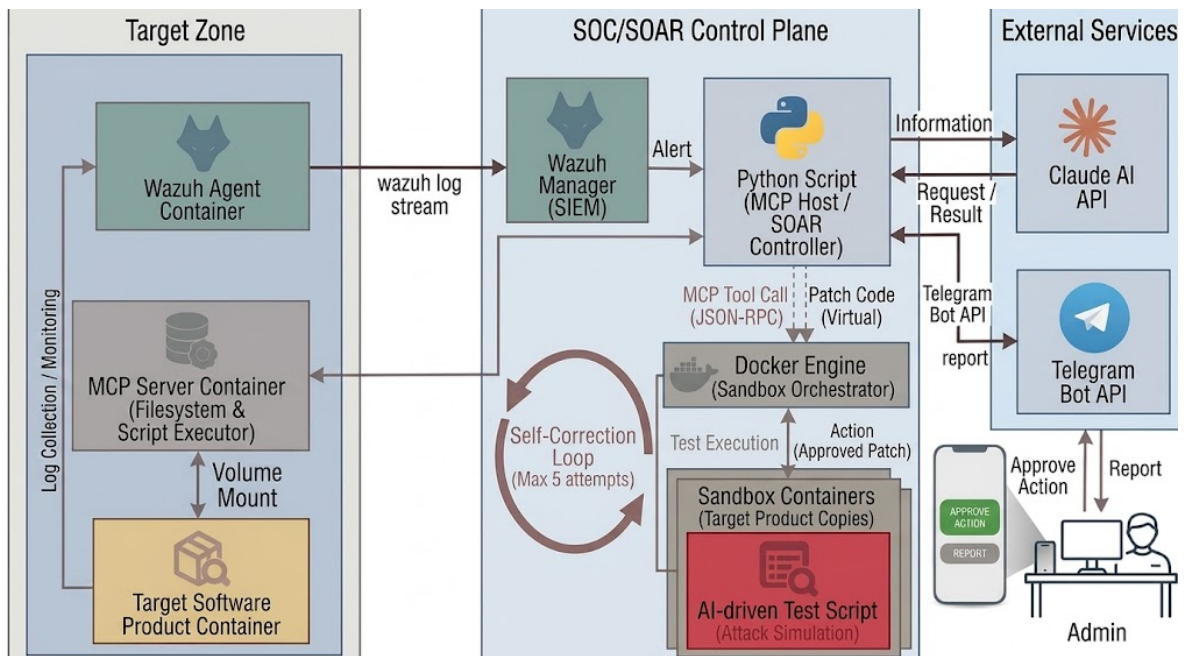


Figure 1: Agentic-SOAR system architecture for automated vulnerability remediation and self-healing security operations.

As illustrated in Figure 1, the architecture follows a centralized control model where the Python Host acts as the orchestration core, while all remediation actions are validated before deployment through sandbox simulation and administrator approval.

The first logical zone is the Target Zone, which contains the production environment and all com-

ponents that must be protected. The central element of this zone is the Target Software Product Container, representing the real application running in production. This container is mounted as a shared volume so that security monitoring and remediation operations can be performed without directly exposing the application to external access.

The Wazuh Agent Container continuously monitors logs, process execution, and suspicious behaviors generated by the target application. Its responsibility is to collect raw security events and forward them as a Wazuh log stream to the centralized SIEM platform for analysis. This ensures continuous visibility into the operational status of the protected system.

Alongside the monitoring component, the MCP Server Container functions as the local execution entity of the architecture. It provides controlled filesystem access and script execution capabilities on the target machine. Through operations such as `read_file` and `write_file`, the MCP Server allows secure retrieval of vulnerable source code and precise deployment of validated patches. This design prevents the AI model from directly interacting with production resources while maintaining sufficient operational capability for remediation.

The second logical zone is the SOC/SOAR Control Plane, which serves as the intelligence and orchestration center of the entire system. Wazuh Manager operates as the SIEM platform and receives log streams from the Wazuh Agent. It analyzes security events using detection rules and correlation logic to identify suspicious behaviors such as SQL Injection attempts, Remote Code Execution payloads, or abnormal system processes. Once a threat is detected, the manager generates a structured JSON alert with severity classification.

At the center of this control plane is the Python Script, implemented as the MCP Host and SOAR Controller. This component acts as the operational heart of the project. It continuously listens for alerts from Wazuh Manager, retrieves contextual source code from the MCP Server, communicates with Claude AI for vulnerability reasoning, and manages the entire patch validation workflow. It also controls the Docker Engine, which is responsible for dynamically creating isolated sandbox containers for safe patch testing.

The Docker Engine functions as the Sandbox Orchestrator. Instead of applying AI-generated patches directly to the production system, the controller creates temporary sandbox containers that replicate the original application environment. This ensures that every remediation attempt is tested under realistic conditions before deployment. The AI-driven test script inside the sandbox simulates the original attack scenario to verify both vulnerability elimination and system stability.

A critical reliability mechanism in this architecture is the Self-Correction Loop. If the generated patch fails during sandbox validation due to syntax errors, runtime failures, or incomplete logic, the error logs are returned to Claude AI for further reasoning. The AI then generates an improved patch

version, and the validation process repeats for up to five iterations. This significantly increases patch reliability and reduces the risk of production disruption.

The third logical zone consists of External Services, which provide intelligence and administrator interaction. Claude AI API functions as the reasoning engine of the system. It receives structured context packages from the Python Host, including security alerts, vulnerable source code, and operational prompts. Based on this information, the AI performs vulnerability analysis, requests additional contextual knowledge if necessary, and designs suitable remediation patches.

Telegram Bot API provides the human-in-the-loop interaction layer. It allows the administrator to receive detailed reports, review sandbox validation results, compare code differences, and approve or reject final deployment actions remotely. This preserves human control over high-risk production modifications while maintaining automation efficiency.

The updated architecture uses direct internal communication between the Python Host and the MCP Server inside the Docker infrastructure. This eliminates unnecessary latency and prevents sensitive source code from passing through intermediate internet services. Only the Python Host is permitted to establish outbound connections to Claude AI and Telegram, while all remaining internal components remain isolated from public internet exposure. This design improves confidentiality, reduces attack surface, and strengthens centralized control.

0.1.2 Integration Workflow of the Proposed System

The operational workflow of the proposed system is organized into four major stages, forming a complete security response lifecycle from attack detection to safe remediation deployment.

The first stage is Detection. The process begins when an attacker exploits a vulnerability in the Target Software Product, such as SQL Injection or Remote Code Execution. Since the application is continuously monitored by the Wazuh Agent, abnormal logs, suspicious requests, or unusual process executions are immediately captured. These security events are forwarded as a Wazuh log stream to the Wazuh Manager.

The SIEM platform analyzes the incoming data using predefined detection rules and behavioral correlation. Once suspicious activity is confirmed, it generates a high-priority JSON alert containing critical information about the incident, including attack type, severity, and system context. This alert serves as the trigger for the automated SOAR workflow.

The second stage is Context Gathering. The Python Host continuously listens for alerts from the SIEM platform. Once an alert is received, it parses the JSON structure to extract key information such as attack type, victim IP address, and most importantly, the suspected vulnerable file path.

This step is essential because remediation must be based on actual application code rather than detection metadata alone.

After identifying the vulnerable target, the Python Host sends a direct request to the MCP Server. The MCP Server executes the `read_file` operation and returns the complete source code of the affected file. The Host then combines the alert information, source code, and remediation instructions into a structured context package that is sent to Claude AI.

At this stage, the system also supports a Knowledge Loop mechanism. If Claude AI determines that a single file is insufficient for safe remediation, it requests additional contextual resources such as configuration files, utility modules, or database-related logic. The Python Host then instructs the MCP Server to retrieve these files. This iterative process continues until the AI has enough understanding of the complete system logic to generate a reliable patch.

The third stage is Simulation and Self-Correction. After sufficient context is collected, Claude AI analyzes the vulnerability and generates the first patch candidate. However, this patch is never deployed directly to production. Instead, the Python Host instructs the Docker Engine to create a sandbox container that acts as an isolated replica of the real target environment.

The generated patch is injected into the sandbox, and an AI-driven test script is executed to simulate the original attack scenario. This allows the system to verify whether the vulnerability has been successfully removed while ensuring that normal system functionality remains unaffected.

If the test fails, whether due to syntax problems, runtime exceptions, or incorrect business logic, the traceback and execution logs are returned to Claude AI. The AI then performs self-correction by analyzing the failure and generating an improved patch version. This validation loop may continue for up to five attempts until the system confirms that the patch is both effective and stable.

If all five remediation attempts fail, the system classifies the incident as a critical unresolved vulnerability. In this case, the Python Host immediately sends a high-priority alert to the administrator through Telegram, including the vulnerability details, failed patch attempts, and sandbox execution logs. Instead of continuing automated remediation, the workflow is terminated and manual intervention is required. The administrator must directly inspect the system, analyze the root cause, and perform corrective actions manually to prevent further operational risks. This mechanism ensures that automation does not introduce unsafe changes when the vulnerability exceeds the system's autonomous repair capability.

The fourth stage is Human-in-the-loop Approval and Final Execution. Once the sandbox validation is successful, the Python Host generates a detailed report including the vulnerability type, testing results, and code differences between the original and patched versions. This report is sent to the administrator through Telegram.

At this point, the administrator chooses whether to approve or reject the remediation action. If approval is granted, the Python Host notifies Claude AI that final deployment is authorized. Before writing to production, the AI performs a final concurrency check by requesting the MCP Server to read the latest version of the real file again. This ensures that no unexpected modifications occurred during the waiting period for administrator approval.

After successful verification, the MCP Server executes the `write_file` operation to replace the vulnerable source code with the validated patch. A final notification is then sent through Telegram confirming that the production system has been securely remediated.

If the administrator rejects the patch, the Python Host immediately initiates cleanup procedures. The sandbox container is deleted, the temporary patch is discarded from memory, and no write operation is performed on the production system. Telegram then sends a confirmation message indicating that the automatic remediation process has been canceled and the original system state remains unchanged.

This workflow ensures that the system achieves not only automated vulnerability remediation but also controlled, auditable, and production-safe deployment. By combining SOAR orchestration, MCP communication, AI reasoning, sandbox validation, and administrator approval, the proposed architecture provides a practical model for intelligent cybersecurity operations with strong reliability and security guarantees.